```
import asyncio
import os
import json
import logging
import tempfile
from decimal import Decimal
from dataclasses import dataclass, asdict
from typing import Dict, List, Optional, Tuple
from contextlib import asynccontextmanager
from dotenv import load_dotenv
from fastapi import FastAPI, Request, Response
from twilio.twiml.messaging_response import MessagingResponse
from twilio.rest import Client
import uvicorn
import ccxt.pro as ccxt_pro
import requests
from openai import OpenAI

load_dotenv()

# --- ENTERPRISE PRODUCTION LOGGING ---
logging.basicConfig(
        level=logging.INFO,
        format='%(asctime)s | [FINAL-COMMAND-CONTROL-SCALPER] | %(levelname)s |
%(message)s',
        handlers=[
        logging.FileHandler("elite_trading_engine.log", encoding="utf-8"),
        logging.StreamHandler()
        ]
)

@dataclass
class Position:
        symbol: str
        entry_price: float
        size: float
        sl_price: float
        tp_price: float
        strategy: str
        highest_price: float
        timestamp: float

@dataclass
class TradeRecord:
```

```
    symbol: str
    entry_price: float
    exit_price: float
    size: float
    pnl_pct: float
    pnl_usd: float
    outcome: str
    strategy: str
    timestamp: float


class RiskManagementSystem:
    def __init__(self, target_goal: float = 100.0, base_risk_pct: float = 0.04, leverage: int = 5):
    self.target_goal = target_goal
    self.base_risk_pct = Decimal(str(base_risk_pct))
    self.leverage = leverage
    self.consecutive_losses = 0

    def calculate_safe_margin(self, free_usdt: float) -> float:
    free_dec = Decimal(str(free_usdt))
    if free_dec <= 0:
    return 0.0
    calculated_margin = free_dec * self.base_risk_pct
    min_safe_margin = max(Decimal("1.2"), Decimal("5.5") / Decimal(str(self.leverage)))
    final_margin = max(calculated_margin, min_safe_margin)
    max_allowed = free_dec * Decimal("0.35")
    return float(min(final_margin, max_allowed))

    def evaluate_circuit_breaker(self, pnl_pct: float) -> bool:
    if pnl_pct < 0:
    self.consecutive_losses += 1
    else:
    self.consecutive_losses = 0
    return self.consecutive_losses >= 3


class VerifiedSignalEngine:
    @staticmethod
    def compute_indicators(ohlcv: List[list]) -> dict:
    if not ohlcv or len(ohlcv) < 25:
    return {}

    closes = [c[4] for c in ohlcv]
    highs = [c[2] for c in ohlcv]
```

```
        lows = [c[3] for c in ohlcv]
        volumes = [c[5] for c in ohlcv]

        tr_list = [max(highs[i]-lows[i], abs(highs[i]-closes[i-1]), abs(lows[i]-closes[i-1])) for i in
range(1, len(ohlcv))]
        atr = sum(tr_list[-7:]) / 7 if len(tr_list) >= 7 else (highs[-1] - lows[-1])

        ema_3 = sum(closes[-3:]) / 3
        ema_9 = sum(closes[-9:]) / 9

        gains, losses = 0.0, 0.0
        for i in range(-7, 0):
        diff = closes[i] - closes[i-1]
        if diff > 0:
                gains += diff
        else:
                losses += abs(diff)
        avg_gain = gains / 7
        avg_loss = losses / 7
        rsi = 100.0 if avg_loss == 0 else 100 - (100 / (1 + (avg_gain / avg_loss)))

        avg_vol = sum(volumes[-10:-1]) / 9 if len(volumes) >= 10 else volumes[-1]
        vol_ratio = volumes[-1] / avg_vol if avg_vol > 0 else 1.0

        return {
        "atr": atr, "ema_3": ema_3, "ema_9": ema_9,
        "rsi": rsi, "vol_ratio": vol_ratio, "current_close": closes[-1]
        }

        @staticmethod
        def verify_and_evaluate_signal(ind: dict) -> Tuple[str, float]:
        if not ind:
        return "NONE", 0.0

        is_ema_bullish = ind["ema_3"] > ind["ema_9"]
        is_rsi_valid = 38 <= ind["rsi"] <= 72
        is_volume_confirmed = ind["vol_ratio"] > 1.35

        if is_ema_bullish and is_rsi_valid and is_volume_confirmed:
        return "Verified 1M Momentum Scalp ⚡", ind["atr"]
        elif ind["rsi"] < 22 and ind["current_close"] >= ind["ema_9"] and ind["vol_ratio"] > 1.2:
        return "Verified 1M Oversold Reversal 🔄", ind["atr"]

        return "NONE", ind.get("atr", 0.0)
```

```
class InstitutionalExchangeHandler:
        def __init__(self):
        is_testnet = os.getenv("BINANCE_TESTNET", "False").lower() == "true"
        self.exchange = ccxt_pro.binance({
        'apiKey': os.getenv("BINANCE_API_KEY"),
        'secret': os.getenv("BINANCE_SECRET_KEY"),
        'enableRateLimit': True,
        'timeout': 15000,
        'options': {'defaultType': 'future', 'adjustForTimeDifference': True}
        })
        if is_testnet:
        self.exchange.set_sandbox_mode(True)
        logging.warning("⚠️ [MODE] Running on BINANCE TESTNET.")
        else:
        logging.info("🟢 [MODE] Running on BINANCE LIVE.")

        async def get_free_usdt(self) -> float:
        try:
        balance = await self.exchange.fetch_balance()
        return float(balance.get('USDT', {}).get('free', 0.0))
        except Exception as e:
        logging.error(f"Balance Fetch Error: {e}")
        return 0.0

        async def fetch_exchange_positions(self) -> dict:
        try:
        positions = await self.exchange.fetch_positions()
        active_map = {}
        for p in positions:
                if float(p.get('contracts', 0)) > 0:
                sym = p['symbol']
                entry = float(p.get('entryPrice', 0))
                active_map[sym] = Position(
                symbol=sym, entry_price=entry, size=float(p.get('contracts', 0)),
                sl_price=entry * 0.985, tp_price=entry * 1.03,
                strategy="Synced Verified Position 🔄", highest_price=entry,
timestamp=time.time()
                )
        return active_map
        except Exception as e:
        logging.error(f"Sync Positions Error: {e}")
        return {}
```

```
        async def execute_verified_order(self, symbol: str, price: float, margin: float, leverage:
int, atr: float, strategy: str) -> Optional[Position]:
        try:
        raw_size = (margin * leverage) / price
        size = float(self.exchange.amount_to_precision(symbol, raw_size))

        notional_value = size * price
        if notional_value < 5.2:
                logging.warning(f"⚠️ Notional value below exchange threshold
({notional_value}). Skipping {symbol}.")
                return None

        try:
                await self.exchange.set_leverage(leverage, symbol)
        except Exception:
                pass

        order = await self.exchange.create_market_buy_order(symbol, size)
        entry = float(order.get('average', price))

        sl = float(self.exchange.price_to_precision(symbol, entry - (atr * 1.0)))
        tp = float(self.exchange.price_to_precision(symbol, entry + (atr * 2.2)))

        try:
                await self.exchange.create_order(symbol, 'STOP_MARKET', 'sell', size, None,
{'stopPrice': sl, 'reduceOnly': True})
                await self.exchange.create_order(symbol, 'TAKE_PROFIT_MARKET', 'sell', size,
None, {'stopPrice': tp, 'reduceOnly': True})
        except Exception as bracket_err:
                logging.error(f"CRITICAL: Bracket Order Failed {symbol}: {bracket_err}.
Emergency Close!")
                await self.close_position(symbol, size)
                return None

        return Position(
                symbol=symbol, entry_price=entry, size=size,
                sl_price=sl, tp_price=tp, strategy=strategy,
                highest_price=entry, timestamp=time.time()
        )
        except Exception as e:
        logging.error(f"Verified Execution Error on {symbol}: {e}")
        return None
```

```
        async def modify_exchange_stop_loss(self, symbol: str, size: str, new_sl: float):
        try:
        orders = await self.exchange.fetch_open_orders(symbol)
        for o in orders:
                if o['type'] == 'STOP_MARKET':
                await self.exchange.cancel_order(o['id'], symbol)
        precision_sl = float(self.exchange.price_to_precision(symbol, new_sl))
        await self.exchange.create_order(symbol, 'STOP_MARKET', 'sell', size, None,
{'stopPrice': precision_sl, 'reduceOnly': True})
        except Exception as e:
        logging.error(f"Trailing SL Update Error {symbol}: {e}")

        async def close_position(self, symbol: str, size: float):
        try:
        await self.exchange.create_market_sell_order(symbol, size)
        except Exception as e:
        logging.error(f"Close Position Error {symbol}: {e}")

        async def close(self):
        try:
        await self.exchange.close()
        except Exception:
        pass


class UltimateMasterEngine:
        def __init__(self):
        self.exchange_handler = InstitutionalExchangeHandler()
        self.risk_manager = RiskManagementSystem()

        self.twilio_client = Client(os.getenv("TWILIO_ACCOUNT_SID"),
os.getenv("TWILIO_AUTH_TOKEN")) if os.getenv("TWILIO_ACCOUNT_SID") else None
        self.openai_client = OpenAI(api_key=os.getenv("OPENAI_API_KEY")) if
os.getenv("OPENAI_API_KEY") else None

        self.symbols = ['BTC/USDT', 'ETH/USDT', 'SOL/USDT', 'XRP/USDT', 'DOGE/USDT']
```